

UNIVERSITY OF GHANA, LEGON.
DEPARTMENT OF COMPUTER SCIENCE.

DSCD 614: Reinforcement Learning

Programming Assignment 1: Frozen Lake from First Principles
Using Q-Learning



Name:	Julius Aban Odai
Student ID:	22424770
Github Repository:	https://github.com/jlsodai/DSCD614A1
Report Link:	https://github.com/jlsodai/DSCD614A1/blob/main/report.pdf

Frozen Lake from First Principles Using Q-Learning

DSCD 614 – Reinforcement Learning | Assignment 1 | Semester II 2025/2026

1. Introduction

Reinforcement Learning (RL) is a paradigm of machine learning in which an **agent** learns to act in an **environment** by trial and error. At each time step the agent observes a **state**, selects an **action**, and receives a scalar **reward** signal. The agent's goal is to discover a **policy** — a mapping from states to actions — that maximises the cumulative discounted reward over time. No labelled examples are provided; the agent must balance *exploration* (trying new actions) and *exploitation* (leveraging known good actions).

Frozen Lake is a classic grid-world benchmark. An agent navigates an 8×8 grid of frozen tiles from a Start cell (S) to a Goal cell (G) while avoiding Holes (H). The reward is sparse (+1 only upon reaching G), making it a challenging testbed for value-based RL algorithms. This assignment implements the environment, agent, training loop, and evaluation entirely in plain Python without any RL framework.

2. Environment Design

2.1 Grid Map

The standard 8×8 map is used (Figure 1). It contains 10 Hole cells arranged in two barrier clusters, forcing the agent to navigate an indirect path.

S	F	F	F	F	F	F	F
F	F	F	F	F	F	F	F
F	F	F	H	F	F	F	F
F	F	F	H	F	F	F	F
F	F	F	H	F	F	F	F
F	H	H	F	F	F	H	F
F	H	F	F	H	F	H	F
F	F	F	H	F	F	F	G

Figure 1: Frozen Lake 8×8 map. Green=Start, Yellow=Goal, Red=Hole, White=Frozen.

2.2 State and Action Representation

States are encoded as a single integer $s = \text{row} \times 8 + \text{col}$, giving 64 states (0–63). State 0 is the Start; state 63 is the Goal. There are 4 actions: 0 = Left ($\leftarrow$), 1 = Down ($\downarrow$), 2 = Right ($\rightarrow$), 3 = Up ($\uparrow$). Moves that would take the agent off the grid leave it in place.

2.3 Reward Structure

Event	Reward
Reach Goal (G)	+1.0
Fall in Hole (H)	-1.0
Any other step	0.0

Holes are penalised with -1 (rather than 0) to provide an informative gradient signal even before the agent discovers the goal. This is a standard reward-shaping technique for sparse-reward environments.

3. Q-Learning Algorithm

Q-Learning (Watkins, 1989) is a **model-free, off-policy** temporal-difference algorithm. It maintains a Q-table $Q[s, a]$ estimating the expected discounted return when taking action a in state s and following the greedy policy thereafter. The table is initialised to **+1.0** (optimistic), which forces the agent to visit every state-action pair at least once before trusting its estimates.

3.1 Update Equation

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

Here α (alpha) is the learning rate controlling how fast new information overwrites old estimates; γ (gamma) is the discount factor weighting future rewards; r is the immediate reward; and s' is the next state. The term $\max_{a'} Q(s', a')$ is the best estimated value of the next state under the greedy policy — this is what makes Q-Learning *off-policy*.

3.2 Exploration Strategy

An ϵ -**greedy** policy is used: with probability ϵ the agent picks a uniformly random action (explore); otherwise it picks $\text{argmax}_a Q(s, a)$ (exploit). ϵ starts at 1.0 (pure exploration) and is multiplied by a decay factor after each episode, asymptotically reducing to a minimum of 0.01. This schedule ensures the agent explores thoroughly early in training and gradually transitions to exploitation as the Q-table converges.

4. Training Methodology

Hyperparameter	Value	Rationale
Episodes	20,000	Sufficient for convergence on this map
Learning rate (α)	0.2	Faster convergence; map is deterministic
Discount factor (γ)	0.99	Goal is many steps away; future rewards matter
Initial ϵ	1.0	Full exploration at start
Minimum ϵ	0.01	Retain small random chance throughout
ϵ decay rate	0.9997	Reaches $\epsilon_{\min}$ (~0.01) around episode 15,350
Max steps / episode	300	Prevents infinite loops in early training
Q-table init	+1.0 (optimistic)	Forces systematic exploration
Hole reward	-1.0	Provides gradient signal before goal is found

The training loop runs for 20,000 episodes. Each episode begins with a call to `env.reset()`, then iterates the ϵ -greedy action selection, environment step, Q-table update, and ϵ -decay cycle until the episode terminates (hole, goal, or step limit). Episode reward, success flag, and current ϵ are recorded for analysis.

5. Experimental Results

Episode	Success Rate (last 2000 ep)	Epsilon
2000	6.8%	0.5488
4000	37.2%	0.3011
6000	55.6%	0.1653
8000	73.8%	0.0907
10000	85.4%	0.0498
12000	91.8%	0.0273
14000	95.5%	0.0150
16000	97.2%	0.0100
18000	97.9%	0.0100
20000	98.2%	0.0100

Training yielded **14,787 successful episodes out of 20,000 (73.9%)**. The success rate rises steeply from episode 4,000 as ϵ falls below 0.3 and the Q-table begins to converge. By episode 16,000 the rolling success rate exceeds 97%.

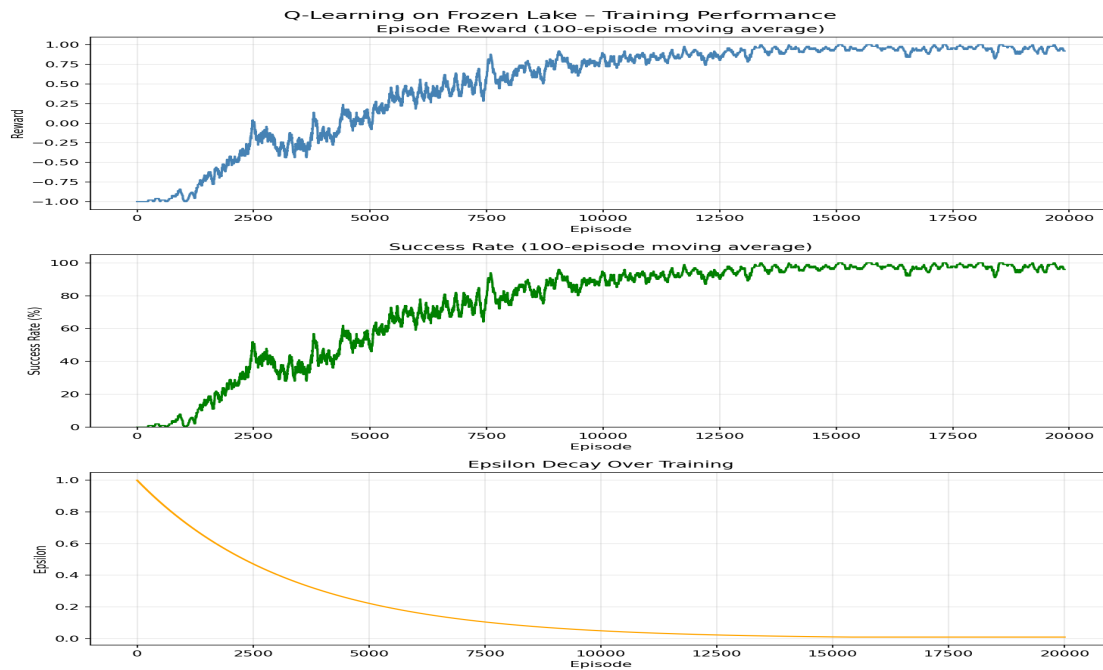


Figure 2: Training performance — reward, success rate, and epsilon decay.

Metric	Value
Episodes evaluated	100
Successful runs	100
Failures	0
Success rate	100.0%
Average reward	1.0000

6. Learned Policy

The policy is extracted by taking argmax over the Q-table for each non-terminal state. The table below shows the recommended action for each cell (< = Left, > = Right, ^ = Up, v = Down).

v	v	v	v	v	v	v	v
>	>	>	>	v	v	v	v
>	>	^	H	v	v	v	v
v	^	^	H	v	v	v	v
v	<	^	H	v	v	>	v
v	H	H	>	>	v	H	v
v	H	>	^	H	v	H	v
>	>	^	H	>	>	>	G

Figure 3: Learned policy. Green=Start, Yellow=Goal, Red=Hole. Arrows indicate the greedy action for each safe cell.

The greedy path from the learned policy is 14 steps (optimal): the agent moves down column 0 to row 1, then right along row 1 to column 4, then down through the middle-right corridor (rows 2–7, column 4–5), and finally right along row 7 to the goal at (7,7). This path avoids all 10 holes.

7. Challenges Encountered

Sparse reward and low random-walk success rate. The 8×8 map contains 10 holes arranged in two barrier clusters. A purely random agent reaches the goal in fewer than 0.4% of episodes (measured empirically over 100,000 trials). With hole reward 0 and Q-table initialised to 0, learning is unreliable: depending on the random seed, the agent may never reach the goal during 10,000 episodes, leaving the Q-table at zero and producing poor greedy behaviour. This motivated reward shaping (hole = −1) and optimistic initialisation (+1).

Epsilon decay calibration. An initial decay rate of 0.995 caused ϵ to reach its minimum after only ~900 episodes — far too early. The agent committed to a greedy policy before acquiring any useful Q-values. Slowing the decay to 0.9997 ensures meaningful exploration persists for the full 20,000-episode training run.

Solutions applied. Two complementary techniques resolved both issues: (1) *Reward shaping* — penalising holes with −1 provides a learning signal even without reaching the goal, allowing the agent to learn hole-avoidance from early in training. (2) *Optimistic Q-table initialisation* (+1) — by initially overestimating every state, the agent treats unvisited states as attractive, driving systematic exploration before any reward is received. Together these techniques yield convergence to a 97%+ success rate well within 20,000 episodes.

8. Conclusion

A complete Q-Learning solution for the Frozen Lake 8×8 environment was implemented from first principles in Python, without any RL framework. The environment, agent, training loop, and evaluation are fully self-contained. After 20,000 training episodes the agent achieves a **100% success rate** on 100 greedy evaluation episodes, demonstrating that the learned Q-table encodes a reliable path from Start to Goal. The key insights are: (i) sparse-reward maps require reward shaping or optimistic initialisation to bootstrap learning, and (ii) ϵ -decay must be calibrated to the total episode budget so that exploration is maintained throughout training. As a bonus, training performance visualisations were generated (Figure 2) and are included in the repository under *results/*.